Computational MR imaging
Laboratory 4: Reconstruction of non-Cartesian k-space data

Code submission is due by 23:55 on Monday. Please upload your code to StudOn in a described format. Late submissions will not be accepted.

Learning objectives

- Reconstruct non-Cartesian MRI data using gridding and NUFFT toolbox
- Apply gridding operations: density compensation, oversampling and deapodization
- Learn to use the NUFFT toolbox

1. Radial sampling pattern:

1.1 Data included in this exercise

- Radial kdata: *load_kdata()*.
- Plot the radial kdata and explore the meaning of each dimension.

1.2 Implement method: *get_traj()* and generate a sampling trajectory that corresponds to this data. **Figure 1** shows a plot of the first 10 spokes of such a trajectory for reference.

- The length of each spoke should be 1 from -0.5 to 0.5.
- The first spoke should start at the angle of $90°$.
- Use *self.PI* and *self.GA* for π and golden angle, respectively.
- Think about a complex number in polar form.

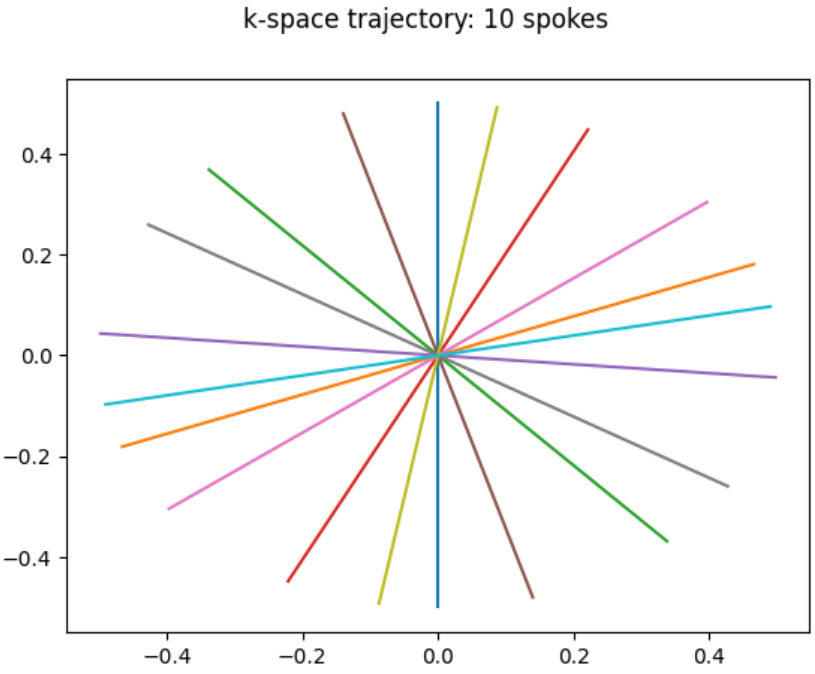


Figure 1: Radial trajectory with golden angle increment. Note that the first angle is at pi/2.

1.3 Implement method: *calc_nyquist()* to calculate the nyquist rate of radial sampling.

2. **Basic gridding reconstruction:**
   2.1 Implement method: *grid_radial()* that maps radial k-space data to Cartesian k-space data
      - Use the provided *grid()* function, in which a triangular gridding kernel of width 2 is used

   2.2 Map the radial k-space data to Cartesian k-space and reconstruct it

   2.3 Discuss the artifacts on the reconstructed image. Can you guess what organ was imaged?

3. **Density compensation**:
   3.1 Implement method: *get_ramp()* to get a ramp filter for density compensation.
      - Consider the minimum and maximum value of each spoke.

   3.2 Implement method: *grid_radial_ds()* to get density compensated Cartesian k-space data.
      - Plot gridded k-space data

   3.3 Reconstruct density compensated Cartesian kdata.

   3.4 Do you need to employ oversampling and de-apodization on this dataset? Explain your answer.

4. **Oversampling**
   4.1 Implement method: *grid_radial_ds_os()* to apply oversampling on density compensated k-space data
      - Think about the matrix size after oversampling

   4.2 Implement method: *center_crop_2d()* to crop the oversampled reconstructed image to its original shape

   4.3 Reconstruct the oversampled data.
      - Plot the gridded oversampled k-space data.
      - Plot the reconstruction of oversampled data.
      - Plot the cropped reconstructed image

5. **De-apodization**
   5.1 Implement method: *get_c()* to get the grinding kernel in image domain
      - Think of the convolution of the delta function

   5.2 Implement method: *deapodization()* that deapodizes the reconstructed image
      - Radial k-space data should be density compensated and oversampled

- Reconstructed image should be cropped to its original size

$$\widehat{m}(x,y) = \frac{\mathbf{1}}{\boldsymbol{c(x,y)+a}}\left\{\left[\left(m(x,y) * s(x,y)\right)c(x,y)\right] * \mathrm{III}\left(\frac{x}{FOV_x}, \frac{y}{FOV_y}\right)\right\}$$

6. NUFFT toolbox

See these links for NUFFT toolbox: Source, Tutorial

6.1 Implement method: *nufft_traj()* to generate a radial trajectory that fits NUFFT toolbox

- The length of each spoke is 2π ($-\pi \sim \pi$)
- The first spoke starts at the angle of $90°$
- Real and Imaginary parts are separated

6.2 Implement method: *nufft_kdata()* to convert the k-space data to the format that fits NUFFT toolbox

- The radial k-space data should be density compensated

6.3 Implement method: *nufft_recon()* that reconstructs radial k-space data

- NUFFT adjoint operator should be defined

6.4 Compare NUFFT reconstructions to gridding reconstructions and discuss results.